

# K230 OpenCV (cv2) API Reference

Based on K230 CanMV firmware, referencing original OpenCV demos and opencv\_demo collection

⚠ K230 specific limitations marked | 📄 Source code examples included

## Color Space Conversion

| Function                    | Signature                            | Example Files |
|-----------------------------|--------------------------------------|---------------|
| <code>cvtColor</code>       | <code>cv2.cvtColor(src, code)</code> | 01, 02, 13    |
| <code>COLOR_BGR2GRAY</code> | BGR→Grayscale, most commonly used    |               |
| <code>COLOR_BGR2HSV</code>  | BGR→HSV, for color segmentation      |               |

### Example

```
# BGR → Grayscale (preprocessing for edge detection, thresholding, etc.)
gray = cv2.cvtColor(img_np, cv2.COLOR_BGR2GRAY)

# BGR → HSV (must convert before inRange color segmentation)
hsv = cv2.cvtColor(img_np, cv2.COLOR_BGR2HSV)
```

## Geometric Transformation

| Function                         | Signature                                                    | Example Files |
|----------------------------------|--------------------------------------------------------------|---------------|
| <code>resize</code>              | <code>cv2.resize(src, (w, h))</code>                         | 01            |
| <code>flip</code>                | <code>cv2.flip(src, 1)</code> 1=horizontal                   | 01            |
| <code>warpAffine</code>          | <code>cv2.warpAffine(src, M, (w, h))</code>                  | 01            |
| <code>getRotationMatrix2D</code> | <code>cv2.getRotationMatrix2D((cx, cy), angle, scale)</code> | 01            |

## Example

```
# Horizontal flip
img_np[:, :, :] = cv2.flip(img_np, 1)

# Rotate 45 degrees + scale 0.7x
M = cv2.getRotationMatrix2D((w//2, h//2), 45, 0.7)
img_np[:, :, :] = cv2.warpAffine(img_np, M, (w, h))

# Downsample for speed (commonly used for heavy operations like HoughCircles/matchTemplate)
small = cv2.resize(img_np, (320, 240))
gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
```

## Thresholding

| Function          | Signature                                                      | Example Files |
|-------------------|----------------------------------------------------------------|---------------|
| threshold         | cv2.threshold(src, thresh, maxval, type)                       | 02, 12        |
| THRESH_BINARY     | >thresh → maxval                                               |               |
| THRESH_OTSU       | Automatically calculate optimal threshold                      |               |
| adaptiveThreshold | cv2.adaptiveThreshold(src, maxval, method, type, blockSize, C) | 12            |

## Example

```
# Fixed threshold 127
ret, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# OTSU automatic threshold (works better when grayscale brightness is uneven)
ret, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_OTSU)

# Adaptive Gaussian threshold (scenarios with local illumination changes)
binary = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, 11, 2)
```

## Filtering (Blurring)

| Function     | Signature                           | Example Files |
|--------------|-------------------------------------|---------------|
| GaussianBlur | cv2.GaussianBlur(src, (k,k), sigma) | 05, 07, 10    |
| blur         | cv2.blur(src, (k,k))                | 10            |
| medianBlur   | cv2.medianBlur(src, k)              | 10            |

| Function        | Signature                                           | Example Files |
|-----------------|-----------------------------------------------------|---------------|
| bilateralFilter | cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace) | 10            |
| filter2D        | cv2.filter2D(src, ddepth, kernel)                   | 10            |

## Example

```
# Gaussian blur for denoising (required before Hough detection)
blur = cv2.GaussianBlur(gray, (7, 7), 1.5)

# Median blur (remove salt-and-pepper noise)
out = cv2.medianBlur(img_np, 7)

# Custom sharpening kernel
sharpen = np.array([[ -1.0, -1.0, -1.0], [-1.0, 9.0, -1.0], [-1.0, -1.0, -1.0]])
out = cv2.filter2D(img_np, -1, sharpen)
```

## Edge Detection

| Function  | Signature                      | Example Files |
|-----------|--------------------------------|---------------|
| Canny     | cv2.Canny(src, low, high)      | 06, 11        |
| Sobel     | cv2.Sobel(src, ddepth, dx, dy) | 11            |
| Laplacian | cv2.Laplacian(src, ddepth)     | 11            |

## Example

```
# Canny dual threshold edge detection
edges = cv2.Canny(gray, 80, 200)

# Grayscale output → display
img_out = image.Image(W, H, image.GRAYSCALE, alloc=image.ALLOC_REF, data=edges)
Display.show_image(img_out)

# Color overlay output (white edges + original image)
edges = cv2.Canny(gray, 80, 200)
img_np[:, :, 0] = cv2.add(img_np[:, :, 0], edges) # B channel
img_np[:, :, 1] = cv2.add(img_np[:, :, 1], edges) # G channel
img_np[:, :, 2] = cv2.add(img_np[:, :, 2], edges) # R channel
Display.show_image(img)
```

# Morphology

| Function                           | Signature                                            | Example Files |
|------------------------------------|------------------------------------------------------|---------------|
| <code>getStructuringElement</code> | <code>cv2.getStructuringElement(shape, (k,k))</code> | 02            |
| <code>erode</code>                 | <code>cv2.erode(src, kernel)</code>                  | 02            |
| <code>dilate</code>                | <code>cv2.dilate(src, kernel)</code>                 | 02            |
| <code>morphologyEx</code>          | <code>cv2.morphologyEx(src, op, kernel)</code>       | 02, 07, 13    |
| <code>MORPH_OPEN</code>            | Opening (denoising)                                  |               |
| <code>MORPH_CLOSE</code>           | Closing (hole filling)                               |               |

## Example

```
# Create structuring elements
kr = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))    # Rectangular kernel
ke = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3)) # Elliptic kernel

# Morphological operations on binary images
ret, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_OTSU)
closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kr)    # Closing for hole filling
opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, ke)     # Opening for denoising
```

# Contour Analysis

| Function                        | Signature                                                           | Example Files |
|---------------------------------|---------------------------------------------------------------------|---------------|
| <code>findContours</code>       | <code>cv2.findContours(src, mode, method)</code>                    | 03, 04, 07    |
| <code>contourArea</code>        | <code>cv2.contourArea(cnt)</code>                                   | 03, 04, 07    |
| <code>arcLength</code>          | <code>cv2.arcLength(cnt, closed)</code>                             | 03, 04, 07    |
| <code>boundingRect</code>       | <code>cv2.boundingRect(cnt)</code>                                  | 03, 07        |
| <code>approxPolyDP</code>       | <code>cv2.approxPolyDP(cnt, epsilon, closed)</code>                 | 03, 04, 07    |
| ⚠ K230 returns (N,2)            | Use <code>pt[0]</code> for index, not <code>pt[0][0]</code>         |               |
| <code>drawContours</code>       | <code>cv2.drawContours(img, contours, idx, color, thickness)</code> | 03, 04, 07    |
| <code>convexHull</code>         | <code>cv2.convexHull(cnt)</code>                                    | 07            |
| <code>minEnclosingCircle</code> | <code>cv2.minEnclosingCircle(cnt)</code>                            | 07            |

| Function             | Signature                     | Example Files |
|----------------------|-------------------------------|---------------|
| <code>moments</code> | <code>cv2.moments(cnt)</code> | 04            |

## Example

```
# Complete contour pipeline
contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cnt_limit = 0
for c in contours:
    area = cv2.contourArea(c)
    if area < 500: continue
    cnt_limit += 1
    if cnt_limit > 50: break          # Upper limit to prevent memory overflow

    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.02 * peri, True) # Polygon approximation

    x, y, w, h = cv2.boundingRect(c)
    cv2.drawContours(img_np, [approx], -1, (0, 255, 0), 2)
    cv2.rectangle(img_np, (x, y), (x + w, y + h), (0, 0, 255), 1)
    cnt_n += 1

# moments for centroid calculation
m = cv2.moments(c)
if m["m00"] > 0:
    cx = int(m["m10"] / m["m00"])
    cy = int(m["m01"] / m["m00"])
```

## Feature Detection

| Function                   | Signature                                                                           | Example Files |
|----------------------------|-------------------------------------------------------------------------------------|---------------|
| <code>HoughLines</code>    | <code>cv2.HoughLines(edges, rho, theta, threshold)</code>                           | 06            |
| ⚠ K230 returns (N,2)       | <code>lines[i,0]=rho, lines[i,1]=theta</code>                                       |               |
| <code>HoughLinesP</code>   | <code>cv2.HoughLinesP(edges, rho, theta, thr, minLen, maxGap)</code>                | 06            |
| <code>HoughCircles</code>  | <code>cv2.HoughCircles(src, method, dp, minDist, param1, param2, minR, maxR)</code> | 05            |
| ⚠ K230 returns (N,3)       | <code>circles[i,0]=x, [i,1]=y, [i,2]=r</code>                                       |               |
| <code>matchTemplate</code> | <code>cv2.matchTemplate(src, templ, method)</code>                                  | 09            |

## Example

```
# Hough circle detection (downsampled for speed)
small = cv2.resize(img_np, (320, 240))
gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (7, 7), 1.5)
circles = cv2.HoughCircles(blur, 3, dp=1, minDist=60, param1=100,
                           param2=70, minRadius=10, maxRadius=100)

if circles is not None:
    for i in range(circles.shape[0]):
        if i >= 30: break
        x = int(circles[i, 0]) * 2    # k230 returns (N,3), not (N,1,3)
        y = int(circles[i, 1]) * 2    # Coordinates x2 back to 640x480
        r = int(circles[i, 2]) * 2
        cv2.circle(img_np, (x, y), 3, (0, 255, 0), -1)
        cv2.circle(img_np, (x, y), r, (0, 0, 255), 2)

# Template matching
rmap = cv2.matchTemplate(binary, templ, cv2.TM_CCOEFF_NORMED)
minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc(rmap)
if maxVal > 0.6:
    cv2.rectangle(img_np, (maxLoc[0], maxLoc[1]),
                  (maxLoc[0] + TS, maxLoc[1] + TS), (0, 255, 0), 2)
```

## Color Processing

| Function             | Signature                                   | Example Files |
|----------------------|---------------------------------------------|---------------|
| <code>inRange</code> | <code>cv2.inRange(src, lower, upper)</code> | 03, 13        |

## Example

```
# HSV color segmentation + green overlay
hsv = cv2.cvtColor(img_np, cv2.COLOR_BGR2HSV)
lo = np.array([35, 60, 60])    # Green HSV lower bound
hi = np.array([85, 255, 255])  # Green HSV upper bound
mask = cv2.inRange(hsv, lo, hi)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, ke)    # Opening for denoising
img_np[:, :, 1] = cv2.add(img_np[:, :, 1], mask)      # Green channel overlay
```

## Arithmetic and Bitwise Operations

| Function         | Signature                        | Example Files |
|------------------|----------------------------------|---------------|
| <code>add</code> | <code>cv2.add(src1, src2)</code> | 13, 15        |

| Function                               | Signature                                             | Example Files |
|----------------------------------------|-------------------------------------------------------|---------------|
| ⚠ Does not support scalar broadcasting | Both arrays must have the same size                   |               |
| <code>absdiff</code>                   | <code>cv2.absdiff(src1, src2)</code>                  | 15            |
| <code>addweighted</code>               | <code>cv2.addweighted(src1, a, src2, b, gamma)</code> | 15            |
| <code>bitwise_not</code>               | <code>cv2.bitwise_not(src)</code>                     | 15            |
| <code>bitwise_and</code>               | <code>cv2.bitwise_and(src1, src2)</code>              | 15            |
| <code>bitwise_or</code>                | <code>cv2.bitwise_or(src1, src2)</code>               | 13, 15        |
| <code>bitwise_xor</code>               | <code>cv2.bitwise_xor(src1, src2)</code>              | 15            |

## Example

```
# Frame difference (motion detection)
if prev_np is not None:
    out = cv2.absdiff(img_np, prev_np)
    img_np[:, :, :] = out

# Inversion (negative effect)
img_np[:, :, :] = cv2.bitwise_not(img_np)

# ⚠ add requires same-size arrays (scalar broadcasting not supported)
# Wrong: cv2.add(img_np, 50)
# Correct: pre-allocate same-size array
add50 = np.zeros((H, w, 3), dtype=np.uint8)
for c in range(w): add50[:, c, :] = 50
out = cv2.add(img_np, add50_3ch)
```

## Pixel Transform

| Function                     | Signature                                               | Example Files |
|------------------------------|---------------------------------------------------------|---------------|
| <code>equalizeHist</code>    | <code>cv2.equalizeHist(src)</code>                      | 14            |
| <code>normalize</code>       | <code>cv2.normalize(src, alpha, beta, norm_type)</code> | 14            |
| <code>convertScaleAbs</code> | <code>cv2.convertScaleAbs(src, alpha, beta)</code>      | 14, 17        |
| <code>LUT</code>             | <code>cv2.LUT(src, lut)</code>                          | 14, 17        |

## Example

```
# Histogram equalization (enhance contrast)
out = cv2.equalizeHist(gray)

# convertScaleAbs: contrast x1.5 brightness -30
out = cv2.convertScaleAbs(gray, alpha=1.5, beta=-30)

# LUT inversion mapping
lut = np.zeros((256, 1), dtype=np.uint8)
for i in range(256): lut[i, 0] = 255 - i
out = cv2.LUT(gray, lut)
```

## Channel Operations

| Function               | Signature                        | Example Files |
|------------------------|----------------------------------|---------------|
| <code>split</code>     | <code>cv2.split(src)</code>      | 16            |
| <code>merge</code>     | <code>cv2.merge(channels)</code> | 13, 15, 16    |
| <code>mean</code>      | <code>cv2.mean(src)</code>       | 16            |
| <code>minMaxLoc</code> | <code>cv2.minMaxLoc(src)</code>  | 09            |

## Example

```
# Split channels, display single channel
b, g, r = cv2.split(img_np)
img_out = image.Image(W, H, image.GRAYSCALE, alloc=image.ALLOC_REF, data=b)
Display.show_image(img_out)

# Channel statistics
avg = cv2.mean(img_np) # Returns (B_mean, G_mean, R_mean)
```

## Drawing

| Function                                                                                          | Example Files |
|---------------------------------------------------------------------------------------------------|---------------|
| <code>line</code> , <code>rectangle</code> , <code>circle</code> , <code>ellipse</code>           | 08            |
| <code>putText</code> , <code>arrowedLine</code> , <code>drawMarker</code> , <code>fillPoly</code> | 08            |



## Example

```
cv2.line(img_np, (50, 80), (250, 80), (0, 255, 0), 3)
cv2.rectangle(img_np, (50, 120), (200, 200), (0, 255, 255), 3)
cv2.circle(img_np, (400, 160), 50, (0, 255, 0), 3)
cv2.ellipse(img_np, (160, 320), (80, 40), 0, 0, 360, (0, 255, 0), 3)
cv2.putText(img_np, "opencv k230", (220, 400),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
cv2.drawMarker(img_np, (400, 260), (0, 255, 0), cv2.MARKER_CROSS, 20, 3)
cv2.arrowedLine(img_np, (300, 80), (500, 80), (255, 0, 0), 3)

# fillPoly for triangle filling
tri = np.array([[80.0, 60.0], [160.0, 60.0], [120.0, 120.0]])
cv2.fillPoly(img_np, [tri], (255, 100, 100), cv2.LINE_AA)
```

## File I/O (REPL exercises, no camera demo)

| Function             | Example                                           |
|----------------------|---------------------------------------------------|
| <code>imread</code>  | <code>img = cv2.imread("/sdcard/test.jpg")</code> |
| <code>imwrite</code> | <code>cv2.imwrite("/sdcard/out.png", img)</code>  |

## Display Method Quick Reference

```
# Color image (after modifying sensor native img, display)
img_np[:, :, :] = cv2.flip(img_np, 1)
Display.show_image(img)

# Grayscale image (cv2 output → image.Image wrapper)
edges = cv2.Canny(gray, 80, 200)
img_out = image.Image(W, H, image.GRAYSCALE, alloc=image.ALLOC_REF, data=edges)
Display.show_image(img_out)

# ⚠ Color images cannot use image.Image(RGB888, ALLOC_REF) to wrap cv2 output!
```